

EECS 753
Prof. Heechul Yun
Date: 2026-05-13

EECS 753 - FINAL PROJECT REPORT

"Task scheduling on FreeRTOS"

1. Introduction

In EECS 753, we have extensively studied real-time (RT) task scheduling, including a variety of RT schedulers (e.g. Rate-Monotonic) and theoretical results which aid in determining whether a given task set is schedulable (e.g. Liu and Layland). We have also acquired practical RT skills through hands-on assignments which involved RT programming on Linux-based systems (Raspberry Pi 5).

However, since I am mostly interested in highly resource constrained embedded systems (e.g. the RP2040 chip, with a very limited 264KB of SRAM), I found it appropriate to learn a RT operating system (RTOS) which specifically targets such platforms. Thus, my final EECS 753 project explores a highly popular open-source RTOS for embedded platforms (FreeRTOS), by designing and deploying a software framework built around it on a resource-constrained microcontroller unit (MCU), the Adafruit Feather RP2040 with RFM95. Moreover, a RT taskset is designed around three external sensor modules sharing a data bus, and the effects of shared resource contention and an attacker task are analyzed.

1.1. Objectives

The following is a comprehensive list of all self-assigned objectives for this final project:

- Schedule a set of tasks in a freely available RTOS for a given target embedded platform.
- Design and implement a task scheduling experiment in an RTOS, including mechanisms to log and process relevant task data (e.g. execution time, deadline misses).
- Measure the real-world effects of shared resource contention on a set of RT tasks, and how they are partially mitigated by applying well-known priority inheritance mechanism (e.g. PIP).
- Design and implement a simple "attacker task" which targets a shared resource, and measure the effects of the attack on other tasks (e.g. increased execution times, induced deadline misses).

2. Hardware

The following is a comprehensive list of all hardware components employed throughout this project, along with relevant specifications, wiring diagrams, and product links.

2.1. Target microcontroller

- **Adafruit Feather RP2040 with RFM95:** An RP2040-based microcontroller board by Adafruit Industries, including 8MB of on-board FLASH memory, plenty of GPIO ports, and a modern USB-C interface. It supports the SPI, UART, and (most relevant here) I²C protocols. For the specifications of the RP2040 microcontroller itself, consult the following entry in this list.

EECS 753
 Prof. Heechul Yun
 Date: 2026-05-13

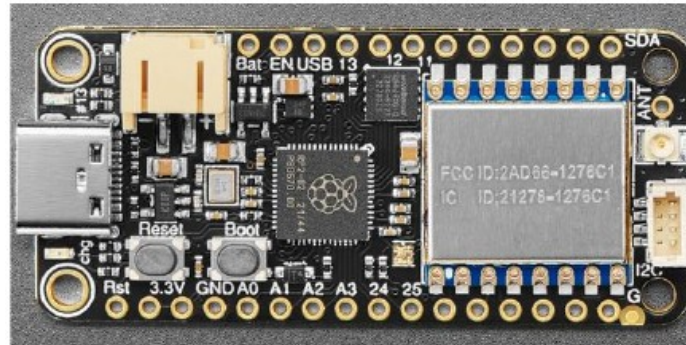


Figure 1: Top-down view of the Adafruit Feather RP2040 with RFM95 microcontroller board. Credit: Adafruit Industries.

- **RP2040 microcontroller chip:** Manufactured by Raspberry Pi, this is low-cost and highly resource-limited chip, including:
 - 32-bit ARM Cortex M0+ dual core CPU, running at 133MHz.
 - 264KB of on-chip SRAM memory.

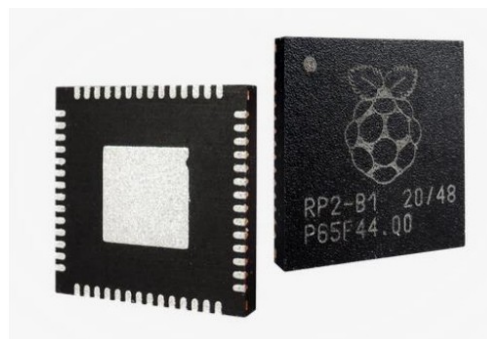


Figure 2: Front and back of the RP2040 microcontroller chip. Credit: Raspberry Pi.

2.2. External sensors

- **Adafruit MMC5603:** A three-axis magnetometer sensor breakout board produced by Adafruit Industries. Generates magnetic field strength data, which can be accessed through the I²C bus.
 - Link: <https://www.adafruit.com/product/5579>
- **Adafruit LSM6DSOX:** A 2-in-1 accelerometer and gyroscope sensor breakout board produced by Adafruit Industries. Generates linear and angular acceleration data in 6 different degrees-of-freedom, which can be accessed through the I²C bus. It also supports the SPI protocol.

EECS 753
 Prof. Heechul Yun
 Date: 2026-05-13

- Link: <https://www.adafruit.com/product/4438>
- **Adafruit BMP390:** A precision barometric pressure sensor and altimeter produced by Adafruit Industries. Generates atmospheric pressure data, which can be accessed (as in the other sensors) through the I²C protocol. It also supports the SPI protocol.
- Link: <https://www.adafruit.com/product/4816>



Figure 3: (Left to right) Top-down images of the Adafruit MMC5603, Adafruit LSM6DOX, and Adafruit BMP390 sensor modules. Credit: Adafruit Industries.

2.3. Hardware wiring diagram

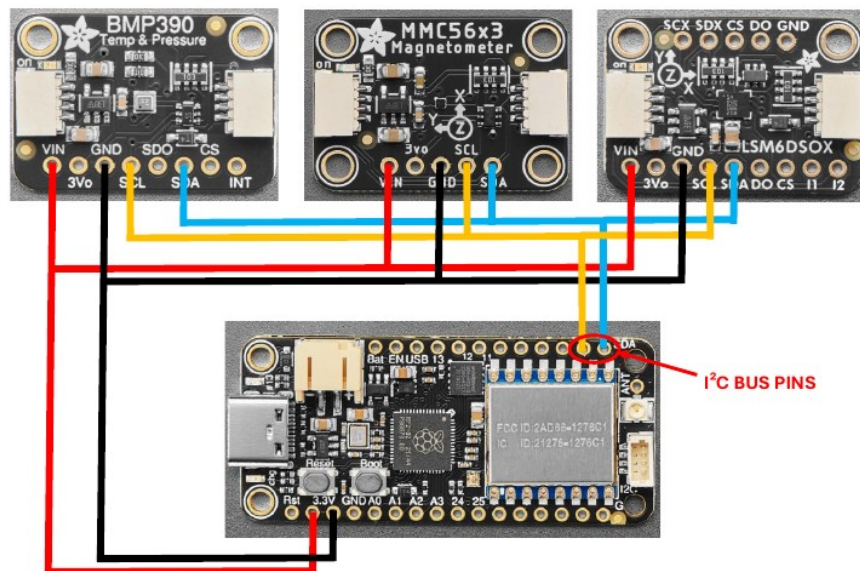


Figure 4: Wiring diagram between the external sensors and the target microcontroller board. The same I²C bus is shared across all sensors.

EECS 753
 Prof. Heechul Yun
 Date: 2026-05-13

3. Software

The following is a comprehensive list of all software components employed throughout this project, along with relevant configuration settings, FreeRTOS task code snippets, among others.

3.1. Project software stack

- **CMake:** Used to generate a Makefile for the project. Required by FreeRTOS and pico-sdk.
 - Link: <https://cmake.org/>
- **FreeRTOS-Kernel:** The standard repository for this project's RTOS of choice.
 - Link: <https://github.com/FreeRTOS/FreeRTOS-Kernel>
- **pico-sdk:** Software development kit for RP2040-based microcontroller boards.
 - Link: <https://github.com/raspberrypi/pico-sdk>
- **arduino-pico:** An open-source Arduino compatibility layer for the pico-sdk.
 - Link: <https://github.com/earlephilhower/arduino-pico>
- **Adafruit_MMC56x3:** Arduino library for the Adafruit MMC5603 sensor module.
 - Link: https://github.com/adafruit/Adafruit_MMC56x3
- **Adafruit_LSM6DS:** Arduino library for the Adafruit LSM6DSOX module.
 - Link: https://github.com/adafruit/Adafruit_LSM6DS
- **Adafruit_BMP3XX_Library:** Arduino library for the Adafruit BMP390 sensor module.
 - Link: https://github.com/adafruit/Adafruit_BMP3XX

3.2. FreeRTOS system configuration

- **FreeRTOSConfig.h relevant settings (not comprehensive):**

#define configUSE_PREEMPTION	1	// Enables task preemption.
#define configTICK_RATE_HZ	((TickType_t) 1000)	// 1000 Hz
#define configMAX_PRIORITIES	32	// Task priorities in range 0-31.
#define configUSE_MUTEXES	1	// Enable mutexes.
#define configUSE_RECURSIVE_MUTEXES	1	// Enable mutexes within mutexes.
#define configUSE_TIME_SLICING	1	// Round-robin for equal prio. tasks.
#define configNUMBER_OF_CORES	2	// rp2040 has 2 cores.
#define configRUN_MULTIPLE_PRIORITIES	0	// Inter-core global task priorities.
#define configUSE_CORE_AFFINITY	1	// Enables pinning tasks to cores.

EECS 753
 Prof. Heechul Yun
 Date: 2026-05-13

3.3. FreeRTOS tasks

The following subsections consistute a comprehensive account of all FreeRTOS tasks designed for this project, their creation parameters, and their implementation code, with relevant annotations.

3.3.1. Magnetometer task

This task periodically reads the magnetic field strength data generated by the magnetometer sensor module, which is connected to the main I²C bus. The I²C bus is a shared resource in this context, which is why accesses to it are protected by a mutex (created with `xSemaphoreCreateMutex` in the main source file, and distributed to all tasks that require it via a shared struct containing a pointer to the mutex). It is assigned a task priority of 3 ("high", among the three sensor tasks), and it is pinned to Core 0 in the RP2040 MCU. Task execution is delayed with `xTaskDelayUntil`, with a period / deadline of 40ms (25Hz). If the `collect_magnetometer_logs` flag is set, its execution data is appended to an array provided by the supervisor task (which later dumps it to the USB serial after the experiment).

```
void createTask_vMagnetometer_UpdateHeading(void){
    TaskHandle_t* task_handle_ptr = &Earendil_Handles->Magnetometer_Handles.task_vMagnetometer_UpdateHeading;
    xTaskCreate(
        vMagnetometer_UpdateHeading,           // Task function
        "vMagnetometer_UpdateHeading",         // Task name (for debugging)
        512,                                   // Task stack depth (in words, NOT bytes!)
        NULL,                                  // Task parameters at creation
        3,                                     // Real-time task priority
        task_handle_ptr                        // Task handle
    );
    vTaskCoreAffinitySet(*task_handle_ptr, 1 <<
        0                                     // Assigned core (options: 0, 1)
    );
}
```

Figure 5: FreeRTOS task parameters for the Magnetometer sensor task.

EECS 753
 Prof. Heechul Yun
 Date: 2026-05-13

**TASK
EXECUTION**

**DATA
LOGGING**

```
void vMagnetometer_UpdateHeading(void* pvParameters){
    (void)pvParameters;    // Parameters unused.

    TickType_t xLastWakeTime = xTaskGetTickCount();
    const TickType_t xFrequency = pdMS_TO_TICKS(PERIOD_VMAGNETOMETER_UPDATEHEADING_MS);
    BaseType_t xWasDelayed;

    while (Earendil_Data->Experimental_Data.experiment_ongoing){
        TickType_t expected_wake_tick = xLastWakeTime + xFrequency;
        xWasDelayed = xTaskDelayUntil(&xLastWakeTime, xFrequency);
        TickType_t actual_wake_tick = xTaskGetTickCount();

        uint64_t start_task_us = time_us_64();

        xSemaphoreTake(Earendil_Mutexes->i2c_mutex, portMAX_DELAY);
        Earendil_Data->Magnetometer_Data.heading_deg = getHeading(hard_calibrated); // Task execution
        xSemaphoreGive(Earendil_Mutexes->i2c_mutex);

        uint32_t task_exec_time_us = (uint32_t)(time_us_64() - start_task_us);

        if (Earendil_Data->Experimental_Data.collect_magnetometer_logs){
            auto& log_array = Earendil_Data->Experimental_Data.magnetometer_logs;
            auto& log_array_idx = Earendil_Data->Experimental_Data.magnetometer_logs_idx;

            if (log_array_idx < Earendil_Experimental::DATA_LOG_SIZE){
                log_array[log_array_idx].exec_time_us = task_exec_time_us;
                log_array[log_array_idx].deadline_missed = (xWasDelayed == pdFALSE);
                log_array[log_array_idx].expected_wake_tick = expected_wake_tick;
                log_array[log_array_idx].actual_wake_tick = actual_wake_tick;
                log_array[log_array_idx].wake_tick_difference = actual_wake_tick - expected_wake_tick;
                log_array_idx++;
            }
        }
    }

    vTaskDelete(NULL);
}
```

Figure 6: Implementation of the Magnetometer task, with relevant sections highlighted. Note that the critical section is wrapped with a FreeRTOS mutex protecting the I²C bus.

3.3.2. Accelerometer & Gyroscope task

This task periodically reads the linear/angular acceleration data generated by the accelerometer and gyroscope sensor module, which is connected to the main I²C bus. Again, the I²C bus is a shared resource, so accesses to it are protected by the mutex. Task execution is delayed with `xTaskDelayUntil`, with a period / deadline of 80ms (12.5Hz). It is assigned a task priority of 2 ("intermediate", among the three sensor tasks), and it is pinned to Core 0 in the RP2040 MCU. Again, if the `collect_accelgyro_logs` flag is set, its execution data is appended to an array provided by the supervisor task.

EECS 753
 Prof. Heechul Yun
 Date: 2026-05-13

```
// =====
// vAccelGyro
// =====

void createTask_vAccelGyro(void){
    TaskHandle_t* task_handle_ptr = &Earendil_Handles->Accelgyro_Handles.task_vAccelGyro;
    xTaskCreate(
        vAccelGyro,                // Task function
        "vAccelGyro",              // Task name (for debugging)
        512,                       // Task stack depth (in words, NOT bytes!)
        NULL,                      // Task parameters at creation
        2,                         // Real-time task priority
        task_handle_ptr            // Task handle
    );
    vTaskCoreAffinitySet(*task_handle_ptr, 1 <<
        0                          // Assigned core (options: 0, 1)
    );
}
```

Figure 7: FreeRTOS task parameters for the Accelerometer & Gyroscope sensor task.

```
void vAccelGyro(void* pvParameters){
    (void) pvParameters;

    (void)pvParameters;    // Parameters unused.

    TickType_t xLastWakeTime = xTaskGetTickCount();
    const TickType_t xFrequency = pdMS_TO_TICKS(PERIOD_VACCELGYRO_MS);
    BaseType_t xWasDelayed;

    while (Earendil_Data->Experimental_Data.experiment_ongoing){
        TickType_t expected_wake_tick = xLastWakeTime + xFrequency;
        xWasDelayed = xTaskDelayUntil(&xLastWakeTime, xFrequency);
        TickType_t actual_wake_tick = xTaskGetTickCount();

        uint64_t start_task_us = time_us_64();

        xSemaphoreTake(Earendil_Mutexes->i2c_mutex, portMAX_DELAY);
        gyroReading();                // Task execution
        xSemaphoreGive(Earendil_Mutexes->i2c_mutex);

        uint32_t task_exec_time_us = (uint32_t)(time_us_64() - start_task_us);

        if (Earendil_Data->Experimental_Data.collect_altimeter_logs){
            auto& log_array = Earendil_Data->Experimental_Data.accelgyro_logs;
            auto& log_array_idx = Earendil_Data->Experimental_Data.accelgyro_logs_idx;

            if (log_array_idx < Earendil_Experimental::DATA_LOG_SIZE){
                log_array[log_array_idx].exec_time_us = task_exec_time_us;
                log_array[log_array_idx].deadline_missed = (xWasDelayed == pdFALSE);
                log_array[log_array_idx].expected_wake_tick = expected_wake_tick;
                log_array[log_array_idx].actual_wake_tick = actual_wake_tick;
                log_array[log_array_idx].wake_tick_difference = actual_wake_tick - expected_wake_tick;
                log_array_idx++;
            }
        }

        vTaskDelete(NULL);
    }
}
```

TASK EXECUTION

DATA LOGGING

Figure 8: Implementation of the Accelerometer & Gyroscope task, with relevant sections highlighted. The critical section is wrapped with a mutex protecting the I²C bus. Minor errata: the flag in "DATA LOGGING" should be collect_accelgyro_logs.

EECS 753
 Prof. Heechul Yun
 Date: 2026-05-13

3.3.3. Altimeter task

This task periodically accesses the barometric pressure and altimetry data generated by the altimeter sensor module, which is connected to the main I²C bus. Again, the I²C bus is a shared resource, so accesses to it are protected by the mutex. Task execution is delayed with *xTaskDelayUntil*, with a period / deadline of 120ms (~8.33Hz). It is assigned a task priority of 1 ("low", among the three sensor tasks), and it is pinned to Core 0 in the RP2040 MCU. Again, if the *collect_altimeter_logs* flag is set, its execution data is periodically appended to an array provided by the supervisor task.

```
// =====
// vAltimeter
// =====

void createTask_vAltimeter(void){
    TaskHandle_t* task_handle_ptr = &Earendil_Handles->Altimeter_Handles.task_vAltimeter;
    xTaskCreate(
        vAltimeter,                // Task function
        "vAltimeter",             // Task name (for debugging)
        512,                      // Task stack depth (in words, NOT bytes!)
        NULL,                     // Task parameters at creation
        1,                        // Real-time task priority
        task_handle_ptr           // Task handle
    );
    vTaskCoreAffinitySet(*task_handle_ptr, 1 <<
        0                        // Assigned core (options: 0, 1)
    );
}
```

Figure 9: FreeRTOS task parameters for the Altimeter sensor task.

EECS 753
 Prof. Heechul Yun
 Date: 2026-05-13

```

void vAltimeter(void* pvParameters){
    (void)pvParameters;    // Parameters unused.

    TickType_t xLastWakeTime = xTaskGetTickCount();
    const TickType_t xFrequency = pdMS_TO_TICKS(PERIOD_VALTIMETER_MS);
    BaseType_t xWasDelayed;

    while (Earendil_Data->Experimental_Data.experiment_ongoing){
        TickType_t expected_wake_tick = xLastWakeTime + xFrequency;
        xWasDelayed = xTaskDelayUntil(&xLastWakeTime, xFrequency);
        TickType_t actual_wake_tick = xTaskGetTickCount();

        uint64_t start_task_us = time_us_64();

        {
            xSemaphoreTake(Earendil_Mutexes->i2c_mutex, portMAX_DELAY);
            altimeterRead();
            xSemaphoreGive(Earendil_Mutexes->i2c_mutex);
            // Task execution
        }

        uint32_t task_exec_time_us = (uint32_t)(time_us_64() - start_task_us);

        if (Earendil_Data->Experimental_Data.collect_altimeter_logs){
            auto& log_array = Earendil_Data->Experimental_Data.altimeter_logs;
            auto& log_array_idx = Earendil_Data->Experimental_Data.altimeter_logs_idx;

            if (log_array_idx < Earendil_Experimental::DATA_LOG_SIZE){
                log_array[log_array_idx].exec_time_us = task_exec_time_us;
                log_array[log_array_idx].deadline_missed = (xWasDelayed == pdFALSE);
                log_array[log_array_idx].expected_wake_tick = expected_wake_tick;
                log_array[log_array_idx].actual_wake_tick = actual_wake_tick;
                log_array[log_array_idx].wake_tick_difference = actual_wake_tick - expected_wake_tick;
                log_array_idx++;
            }
        }
    }

    vTaskDelete(NULL);
}
  
```

Figure 10: Implementation of the Altimeter task, with relevant sections highlighted. The critical section is wrapped with a mutex protecting the I²C bus.

3.3.4. Shared resource attacker task

This attacker task implements a simplified shared resource contention attack, targeting the I²C bus shared by the three sensor modules. It assumes unrestricted access to the I²C mutex. As the attacker task, it is not delayed by `xTaskDelayUntil` (or by any other means), to maximize the load. It is assigned a task priority of 0 (the lowest possible), and it is pinned to Core 1 in the RP2040 MCU. Even using these parameters, the attack nevertheless manages to have an impact on other tasks (see Section 4).

EECS 753
 Prof. Heechul Yun
 Date: 2026-05-13

```
// =====
// vI2CBusHog
// =====

void createTask_vI2CBusHog(){
    TaskHandle_t* task_handle_ptr = &Earendil_Handles->Experimental_Handles.task_vI2CBusHog;
    xTaskCreate(
        vI2CBusHog,                // Task function
        "vI2CBusHog",             // Task name (for debugging)
        512,                      // Task stack depth (in words, NOT bytes!)
        NULL,                     // Task parameters at creation
        0,                        // Real-time task priority
        task_handle_ptr            // Task handle
    );
    vTaskCoreAffinitySet(*task_handle_ptr, 1 <<
        1                          // Assigned core (options: 0, 1)
    );
}
```

Figure 11: FreeRTOS task parameters for the I²C bus hog attacker task.

```
void vI2CBusHog(void* pvParameters){
    (void)pvParameters;

    volatile uint32_t x = 0;

    while (1){
        xSemaphoreTake(Earendil_Mutexes->i2c_mutex, portMAX_DELAY);
        // Hold the mutex for a while
        for (int i = 0; i < 400000; i++){
            x += i;
        }
        xSemaphoreGive(Earendil_Mutexes->i2c_mutex);
    }
}
```

Figure 12: Implementation of the I²C bus hog attacker task. Note how the I²C bus mutex is held by the attacker for a disproportionate amount of cycles by means of a simple for-loop, performing "useless work" for 400,000 iterations.

3.3.5. Supervisor task

This task controls the duration of the task scheduling experiment. By simply disabling a shared flag *experiment_ongoing*, this task signals all sensor tasks that the experiment has ended. As a result, the sensor tasks suspend themselves to prevent further execution data from being produced. Here, the experiment is set to last for 30,000ms (i.e. 30 seconds). After the experiment is over, the supervisor tasks dumps all log data directly to the USB serial in CSV format. These data may be saved to a plain text file within a Linux computer with the command `sudo cat /dev/ttyACM0 | tee log_output.txt`. The shared flags *collect_[SENSOR]_logs* enable / disable log collection for that sensor. Note that this is a helper task that only runs once, and then deletes itself after all execution data has been dumped.

EECS 753
 Prof. Heechul Yun
 Date: 2026-05-13

```
// =====
// vExperimentSupervisor
// =====

void createTask_vExperimentSupervisor(){
    TaskHandle_t* task_handle_ptr = &Earendil_Handles->Experimental_Handles.task_vExperimentSupervisor;
    xTaskCreate(
        vExperimentSupervisor,           // Task function
        "vExperimentSupervisor",         // Task name (for debugging)
        512,                             // Task stack depth (in words, NOT bytes!)
        NULL,                             // Task parameters at creation
        30,                               // Real-time task priority
        task_handle_ptr                  // Task handle
    );
    vTaskCoreAffinitySet(*task_handle_ptr, 1 <<
        0                                // Assigned core (options: 0, 1)
    );
}
```

Figure 13: FreeRTOS task parameters for the supervisor task.

```
void vExperimentSupervisor(void* pvParameters){
    (void)pvParameters;

    Earendil_Data->Experimental_Data.collect_magnetometer_logs = false; // Enable Earendil_Magnetometer logs.
    Earendil_Data->Experimental_Data.collect_accelgyro_logs     = false; // Enable Earendil_AccelGyro logs.
    Earendil_Data->Experimental_Data.collect_altimeter_logs    = false; // Enable Earendil_Altimeter logs.

    Earendil_Data->Experimental_Data.experiment_ongoing = true; // Start experiment.
    vTaskDelay(pdMS_TO_TICKS(EXPERIMENT_DURATION_MS)); // Wait for experiment to conclude.

    Earendil_Data->Experimental_Data.experiment_ongoing = false; // Stop experiment.
    vTaskDelay(pdMS_TO_TICKS(DELAY_AFTER_EXPERIMENT_MS)); // Wait a bit for all tasks to stop.

    if (Earendil_Data->Experimental_Data.collect_magnetometer_logs){
        dumpLogs(
            "magnetometer",
            Earendil_Data->Experimental_Data.magnetometer_logs,
            Earendil_Data->Experimental_Data.magnetometer_logs_idx
        ); // Dump Earendil_Magnetometer logs to USB serial via printf.
    }
    if (Earendil_Data->Experimental_Data.collect_accelgyro_logs){
        dumpLogs(
            "accelgyro",
            Earendil_Data->Experimental_Data.accelgyro_logs,
            Earendil_Data->Experimental_Data.accelgyro_logs_idx
        ); // Dump Earendil_AccelGyro logs to USB serial via printf.
    }
    if (Earendil_Data->Experimental_Data.collect_altimeter_logs){
        dumpLogs(
            "altimeter",
            Earendil_Data->Experimental_Data.altimeter_logs,
            Earendil_Data->Experimental_Data.altimeter_logs_idx
        ); // Dump Earendil_Altimeter logs to USB serial via printf.
    }

    vTaskDelete(NULL);
}
```

**DATA
OUTPUT
TO USB
SERIAL**

Figure 14: Implementation of the Supervisor task, with relevant sections highlighted.

EECS 753
 Prof. Heechul Yun
 Date: 2026-05-13

4. Experimental results

4.1. Task setup parameters

Table 1: Summary of task parameters for the experiment (excluding attackers). In FreeRTOS, a higher value corresponds to a higher task priority (here 31 is the highest).

	Supervisor task	Magnetometer task	Accel&Gyro task	Altimeter task
T := Period/Deadline (ms)	N/A	40	80	120
Assigned core	0	0	0	0
Priority	30	3	2	1

4.2. Running tasks "solo"

Table 2: Results when running each of the sensor tasks in isolation ("solo").

	Magnetometer task	Accel&Gyro task	Altimeter task
T := Period/Deadline (us)	40000	80000	120000
Average execution time (us)	13257	1749	70
Max. exec. time (us)	13493	2421	414
Execution time standard deviation (us)	236	46	30
Deadlines missed	0	0	0

4.3. Running tasks concurrently

Table 3: Results when running all sensor tasks with the parameters given in Table 1. The effects of shared resource contention visible here are explained in Section 5.

	Magnetometer task	Accel&Gyro task	Altimeter task
T := Period/Deadline (us)	40000	80000	120000
Average execution time (us)	13093	8064	14265
Max. exec. time (us)	13372	14352	14474
Execution time standard deviation (us)	338	6153	33
Deadlines missed	0	0	0

EECS 753
 Prof. Heechul Yun
 Date: 2026-05-13

4.4. Shared resource contention attack (I^2C bus)

Table 4: Results when scheduling the *vI2CBusHog* attacker task alongside the sensor tasks, with task parameters as given in Section 3. See Section 5 for a brief discussion.

	Magnetometer task	Accel&Gyro task	Altimeter task
T := Period/Deadline (us)	40000	80000	120000
Average execution time (us)	28734	17260	17130
Max. exec. time (us)	46084	27704	27661
Execution time standard deviation (us)	10490	5079	5862
Deadlines missed	138	0	0

5. Discussion

In this Section, we analyze and interpret the experimental results given in Section 4, which include data for three experiments: running each sensor task in isolation; running all tasks concurrently; and running all tasks concurrently, along with the shared resource contention attacker task.

5.1. Running tasks solo

After running all sensor tasks individually (i.e. "solo"), we arrive at the experimental results detailed in Table 2. Using the execution data, we estimate the taskset utilization to be

$$U = \sum_i \frac{C_i}{T_i} \approx \frac{\tilde{C}_{mag}}{T_{mag}} + \frac{\tilde{C}_{acc}}{T_{acc}} + \frac{\tilde{C}_{alt}}{T_{alt}} \approx 0.371 < LLB(\infty) = \ln(2)$$

where C_i , T_i , and $\tilde{C}_i$ are the WCET, task period, and estimated WCET for task i , respectively. Since the real WCET requires extensive program analysis, we directly approximate it as $\tilde{C}_i = \max(exec\ time_i)$, which corresponds to the third row of the data table. Our approximation suggests that, in principle, this task set should be schedulable. However, we must note that since the data employed for this estimate was obtained by running each task in isolation, it does not account for the effects of shared resource contention. The real utilization estimate is significantly higher in practice (see Section 5.2).

5.2. Running tasks concurrently

After running all sensor tasks concurrently (on the same core), we arrive at the experimental results detailed in Table 3. Using the execution data, we estimate the taskset utilization to be

EECS 753
 Prof. Heechul Yun
 Date: 2026-05-13

$$U = \sum_i \frac{C_i}{T_i} \approx \frac{\tilde{C}_{mag}}{T_{mag}} + \frac{\tilde{C}_{acc}}{T_{acc}} + \frac{\tilde{C}_{alt}}{T_{alt}} \approx 0.634 < LLB(\infty) = \ln(2)$$

where C_i , T_i , and $\tilde{C}_i$ are the WCET, task period, and estimated WCET for task i , respectively. Since the real WCET requires extensive program analysis, we simply approximate it as $\tilde{C}_i = \max(exec\ time_i)$, which corresponds to the third row in the data table. This is the same procedure as in Section 5.1.

When running the three sensor tasks concurrently, we observe that our approximation for the utilization U still does not exceed the Liu and Layland bound of $\ln(2)$, which implies the taskset is in principle schedulable. Although this is based on estimates (and thus is not a proof of schedulability), it is in agreement with the execution data in Table 3, which indicates zero deadlines were missed.

Nevertheless, the effects of I²C bus contention are totally clear when looking at the data. Lower priority tasks suffer from significantly longer execution times compared to their "solo" run (see Tables 2 and 3). Interestingly, the highest priority task (Magnetometer) is not slowed down. Since, by default, mutexes do not respect task priorities, all tasks should suffer from comparable execution delays due to I²C bus contention. The only plausible explanation is the presence of some form of priority inheritance mechanism, which would partially protect the highest priority task from the effects of contention. The FreeRTOS documentation confirms that mutexes use a form of PIP, confirming our suspicions.

5.3. Introducing the attacker task

After running all sensor tasks along with the attacker task, we arrive at the experimental results detailed in Table 4. Using the execution data, we estimate the taskset utilization to be

$$U = \sum_i \frac{C_i}{T_i} \approx \frac{\tilde{C}_{mag}}{T_{mag}} + \frac{\tilde{C}_{acc}}{T_{acc}} + \frac{\tilde{C}_{alt}}{T_{alt}} \approx 1.729 > 1$$

where C_i , T_i , and $\tilde{C}_i$ are the WCET, task period, and estimated WCET for task i , respectively. Since the real WCET requires extensive program analysis, we simply approximate it as $\tilde{C}_i = \max(exec\ time_i)$, which corresponds to the third row in the data table. This is the same procedure as in Section 5.2.

When adding the attacker task, we observe that our new estimate for the sensor task utilization U exceeds not only the Liu and Layland bound of $\ln(2)$, but 1. This strongly implies the sensor taskset is not schedulable. This is in agreement with the data in Table 4, which indicates that the Magnetometer task missed hundreds of deadlines. Even though the attacker task had the lowest possible priority and was scheduled on a different core to all other tasks, it managed to significantly increase execution times for all tasks involved. This result highlights the limitations of priority inheritance methods (such as PIP), which fail to mitigate shared resource contention attacks when the resource load is excessive.

EECS 753
Prof. Heechul Yun
Date: 2026-05-13

6. Conclusion

From the results provided in Section 4 and the analysis provided in Section 5, we conclude that all project objectives (see Section 1.1) have been met. This includes: scheduling a task set in an RTOS; setting up a controlled experiment to experimentally obtain execution data for the taskset; determining the real effects of shared resource contention based off the obtained data; and evaluating the effects of a simplified attack on a shared resource on the taskset. Furthermore, we have experimentally verified that FreeRTOS mutexes do in fact incorporate a form of priority inversion (most likely PIP), as advertised in the documentation page. Finally, we have experimentally demonstrated the limitations of priority inversion mechanisms at counteracting the effects of attacks which hog shared resources.